FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
0001 /**
0002  * @license
0003  * SPDX-License-Identifier: Apache-2.0
0004  */
0005
0006 import React, { useState, useEffect, useRef } from "react";
0007 import {
0008   Sparkles,
0009   Info,
0010   ChevronRight,
0011   BookOpen,
0012   MapPin,
0013   Calendar,
0014   Clock,
0015   Compass,
0016   Layers,
0017   Heart,
0018   Droplet,
0019   Flame,
0020   ShieldAlert,
0021   Instagram,
0022   UserCheck,
0023   CheckCircle2,
0024   RefreshCw,
0025   Copy,
0026   Eye,
0027   Check,
0028   Printer,
0029   Download,
0030   Mic,
0031   MicOff,
0032   Share2,
0033   Send,
0034   X,
0035   Loader2,
0036   Volume2,
0037   VolumeX,
0038   Play,
0039   Pause,
0040   Music,
0041   Trash2,
0042   Bookmark,
0043   Smartphone,
0044   ChevronLeft,
0045   Database,
0046   Search,
0047   Filter,
0048   CreditCard,
0049   Wallet,
0050   Lock,
0051   Star,
0052   ExternalLink
```

FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
0053 } from "lucide-react";
0054 import {
0055   CHAKRAS_REFERENCE,
0056   REFERENCE_SYMBOLS,
0057   PLACEMENT_PRESETS,
0058   ChakraReference,
0059   SpiritualSymbol
0060 } from "./data";
0061 import {
0062   CHAKRA_BASES,
0063   ASTROLOGY_GLYPHS,
0064   PLANETARY_SIGILS,
0065   generateSigilSignificance
0066 } from "./sigilData";
0067 import BirthPlaceMapModal, { WORLD_CITIES, WorldCity } from "./components/BirthPlaceMapMo...
0068 import AuraInkLogo from "./components/AuraInkLogo";
0069 import * as PremiumData from "./premiumData";
0070
0071 const formatBirthDate = (dateStr: string) => {
0072   if (!dateStr) return "";
0073   try {
0074     const d = new Date(dateStr);
0075     return d.toLocaleDateString("en-US", { day: "numeric", month: "long", year: "numeric"...
0076   } catch {
0077     return dateStr;
0078   }
0079 };
0080
0081 const formatBirthTime = (timeStr: string, isApprox: boolean, approxPeriod: string) => {
0082   if (isApprox) {
0083     switch (approxPeriod) {
0084       case "morning": return "Morning (Approx. 08:00 AM)";
0085       case "afternoon": return "Afternoon (Approx. 02:00 PM)";
0086       case "evening": return "Evening (Approx. 06:00 PM)";
0087       case "night": return "Night (Approx. 11:00 PM)";
0088       default: return "Unknown Time (Approx. 12:00 PM)";
0089     }
0090   }
0091   if (!timeStr) return "12:00 PM";
0092   try {
0093     const [hoursStr, minutesStr] = timeStr.split(":");
0094     const hours = parseInt(hoursStr, 10);
0095     const minutes = parseInt(minutesStr, 10);
0096     if (isNaN(hours) || isNaN(minutes)) return timeStr;
0097     const ampm = hours >= 12 ? "PM" : "AM";
0098     const displayHours = hours % 12 === 0 ? 12 : hours % 12;
0099     const displayMinutes = minutes < 10 ? `0${minutes}` : minutes;
0100     return `${displayHours}:${displayMinutes} ${ampm}`;
0101   } catch {
0102     return timeStr;
0103   }
0104 };
```

FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
0105
0106 const getSanskritSign = (signName: string) => {
0107   const map: Record<string, string> = {
0108     aries: "Mesha",
0109     taurus: "Vrishabha",
0110     gemini: "Mithuna",
0111     cancer: "Karka",
0112     leo: "Simha",
0113     virgo: "Kanya",
0114     libra: "Tula",
0115     scorpio: "Vrishchika",
0116     sagittarius: "Dhanu",
0117     capricorn: "Makara",
0118     aquarius: "Kumbha",
0119     pisces: "Meena"
0120   };
0121   return map[signName.toLowerCase()] || signName;
0122 };
0123
0124 const getNakshatraForBirth = (birthDate: string) => {
0125   if (!birthDate) return "Shravana";
0126   const index = (new Date(birthDate).getDate() || 12) % 27;
0127   const NAKSHATRAS = [
0128     "Ashwini", "Bharani", "Krittika", "Rohini", "Mrigashira", "Ardra",
0129     "Punarvasu", "Pushya", "Ashlesha", "Magha", "Purva Phalguni",
0130     "Uttara Phalguni", "Hasta", "Chitra", "Swati", "Vishakha",
0131     "Anuradha", "Jyeshtha", "Mula", "Purva Ashadha", "Uttara Ashadha",
0132     "Shravana", "Dhanishta", "Shatabhisha", "Purva Bhadrapada",
0133     "Uttara Bhadrapada", "Revati"
0134   ];
0135   return NAKSHATRAS[index];
0136 };
0137
0138 const getRulingPlanetForLagna = (lagnaSign?: string) => {
0139   const sign = lagnaSign ? lagnaSign.toLowerCase() : "virgo";
0140   const map: Record<string, string> = {
0141     aries: "Mars (Mangal)",
0142     taurus: "Venus (Shukra)",
0143     gemini: "Mercury (Budha)",
0144     cancer: "Moon (Chandra)",
0145     leo: "Sun (Surya)",
0146     virgo: "Mercury (Budha)",
0147     libra: "Venus (Shukra)",
0148     scorpio: "Pluto / Mars (Ketu)",
0149     sagittarius: "Jupiter (Guru)",
0150     capricorn: "Saturn (Shani)",
0151     aquarius: "Saturn / Uranus (Rahu)",
0152     pisces: "Jupiter / Neptune"
0153   };
0154   return map[sign] || "Mercury (Budha)";
0155 };
0156
```

FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
0157 const getLifePathNumber = (dateStr: string) => {
0158   if (!dateStr) return 7;
0159   const digits = dateStr.replace(/\D/g, "");
0160   let sum = digits.split("").reduce((acc, val) => acc + parseInt(val), 0);
0161   while (sum > 9 && sum !== 11 && sum !== 22 && sum !== 33) {
0162     sum = sum.toString().split("").reduce((acc, val) => acc + parseInt(val), 0);
0163   }
0164   return sum;
0165 };
0166
0167 const getPlanetaryMantra = (signName: string) => {
0168   if (!signName) {
0169     return {
0170       sanskrit: "•  (’é  “i?“Y>’ò"À
0171       phonetics: '"Om Namah Shivaya"',
0172       meaning: "This primal chant triggers total dissolution of obstacles, grounding you ...
0173     };
0174   }
0175   const sign = signName.toLowerCase();
0176   if (sign.includes("sagittarius") || sign.includes("jupiter") || sign.includes("dhanu")) {
0177     return {
0178       sanskrit: "•  (’éK ’Ù “Y$”r 5“é8” &”y5“é/",
0179       phonetics: '"Om Namo Bhagavate Vasudevaya"',
0180       meaning: "This mantra brings celestial wisdom, protection, and boundless material e...
0181     };
0182   }
0183   if (sign.includes("leo") || sign.includes("sun") || sign.includes("simha")) {
0184     return {
0185       sanskrit: "•   ”9#“ò 8”)0”Ù/“é/ ’‰.•2"À
0186       phonetics: '"Om Ghrini Suryaya Namah"',
0187       meaning: "This mantra invokes solar vitality, sovereign power, and radiating confid...
0188     };
0189   }
0190   if (sign.includes("cancer") || sign.includes("moon") || sign.includes("karka")) {
0191     return {
0192       sanskrit: "•  8”¹  “‰K’é>’ò (’é ",
0193       phonetics: '"Om Som Somaya Namah"',
0194       meaning: "This mantra calms emotional waves, soothing stress and nurturing deep int...
0195     };
0196   }
0197   if (sign.includes("virgo") || sign.includes("gemini") || sign.includes("mercury") || si...
0198     return {
0199       sanskrit: "•  ,”   ’ÉA’y>’ò (’é ",
0200       phonetics: '"Om Bum Budhaya Namah"',
0201       meaning: "This mantra activates mental clarity, articulate speech, and cognitive ba...
0202     };
0203   }
0204   if (sign.includes("taurus") || sign.includes("libra") || sign.includes("venus") || sign...
0205     return {
0206       sanskrit: "•  6”   “iA’YM“ >’ò (’é ",
0207       phonetics: '"Om Shum Shukraya Namah"',
0208       meaning: "This mantra attracts material beauty, artistic luxury, and romantic heali...
```

```
    };
  }
  if (sign.includes("capricorn") || sign.includes("aquarius") || sign.includes("saturn") ...
    return {
      sanskrit: "•  6•" 6’‰H“iM‘©0“é/ ’‰.•2"À
      phonetics: '"Om Sham Shanaishcharaya Namah"',
      meaning: "This mantra aligns skeletal structure, unlocking patience, discipline, an...
    };
  }
  return {
    sanskrit: "•  (’é  “i?“Y>’ò"À
    phonetics: '"Om Namah Shivaya"',
    meaning: "This primal chant triggers total dissolution of obstacles, grounding you in...
  };
};

const getMockupBodyOutline = (type: string) => {
  if (type === "back") {
    return (
      <svg viewBox="0 0 100 100" fill="none" stroke="#A37F43" strokeWidth="0.8" opacity="...
        <path d="M 38 15 C 38 30, 42 38, 30 45 C 42 50, 58 50, 70 45 C 58 38, 62 30, 62 1...
        <path d="M 30 45 C 15 50, 5 62, 0 85 M 70 45 C 85 50, 95 62, 100 85" strokeLineca...
        <line x1="50" y1="46" x2="50" y2="100" strokeDasharray="2 3" />
        <path d="M 28 62 C 38 68, 62 68, 72 62" strokeDasharray="2 2" />
      </svg>
    );
  }
  if (type === "chest") {
    return (
      <svg viewBox="0 0 100 100" fill="none" stroke="#A37F43" strokeWidth="0.8" opacity="...
        <path d="M 32 15 C 37 32, 63 32, 68 15" strokeLinecap="round" />
        <path d="M 50 32 Q 30 29, 12 36 M 50 32 Q 70 29, 88 36" strokeLinecap="round" />
        <path d="M 12 36 C 8 55, 20 85, 50 85 C 80 85, 92 55, 88 36" strokeLinecap="round...
        <circle cx="38" cy="62" r="1.5" fill="#A37F43" opacity="0.5" />
      </svg>
    );
  }
  return (
    <svg viewBox="0 0 100 100" fill="none" stroke="#A37F43" strokeWidth="0.8" opacity="0....
      <path d="M 15 20 C 35 15, 62 25, 68 45 C 75 65, 72 85, 68 100" strokeLinecap="round...
      <path d="M 15 20 C 0 45, 12 75, 18 100" strokeLinecap="round" />
      <line x1="42" y1="20" x2="42" y2="100" strokeDasharray="2 3" />
    </svg>
  );
};

const getSacredTattooSvg = () => {
  return `<svg viewBox="0 0 100 100" fill="none" stroke="#A37F43" stroke-width="1.2">
    <g opacity="0.9">
      <path d="M 38 22 A 12 12 0 0 1 62 22" stroke-width="1.2" />
      <path d="M 42 22 A 8 8 0 0 1 58 22" stroke-width="0.8" opacity="0.6" stroke-dasharr...
      <line x1="50" y1="10" x2="50" y2="4" stroke-width="1.2" />
```

```
      <line x1="44" y1="12" x2="41" y2="6" />
      <line x1="56" y1="12" x2="59" y2="6" />
      <line x1="39" y1="16" x2="34" y2="11" />
      <line x1="61" y1="16" x2="66" y2="11" />
      <line x1="36" y1="22" x2="30" y2="20" />
      <line x1="64" y1="22" x2="70" y2="20" />
      <circle cx="50" cy="2" r="0.8" fill="currentColor" />
      <circle cx="39.5" cy="4" r="0.6" fill="currentColor" />
      <circle cx="60.5" cy="4" r="0.6" fill="currentColor" />
    </g>
    <g transform="translate(0, 16)">
      <circle cx="50" cy="30" r="18" stroke-width="1.2" />
      <circle cx="50" cy="30" r="16.5" stroke-dasharray="1.5 1.5" stroke-width="0.7" opac...
      <circle cx="50" cy="30" r="21.5" stroke-width="0.8" opacity="0.4" />
      <polygon points="50,46 34,21 66,21" stroke-width="1.2" />
      <polygon points="50,41 37,25 63,25" stroke-width="0.9" opacity="0.8" />
      <polygon points="50,14 34,39 66,39" stroke-width="1.2" />
      <polygon points="50,19 37,35 63,35" stroke-width="0.9" opacity="0.8" />
      <rect x="26" y="6" width="48" height="48" rx="1" stroke-width="0.5" stroke-dasharra...
    </g>
    <g opacity="0.9">
      <path d="M 50 63 C 45 53, 45 44, 50 44 C 55 44, 55 53, 50 63 Z" stroke-width="1.2" ...
      <path d="M 50 63 C 40 55, 34 47, 43 44 C 47 46, 48 55, 50 63 Z" stroke-width="1" />
      <path d="M 50 63 C 60 55, 66 47, 57 44 C 53 46, 52 55, 50 63 Z" stroke-width="1" />
      <path d="M 50 63 C 32 58, 28 50, 36 46 C 42 48, 45 57, 50 63 Z" stroke-width="0.8" ...
      <path d="M 50 63 C 68 58, 72 50, 64 46 C 58 48, 55 57, 50 63 Z" stroke-width="0.8" ...
    </g>
    <g opacity="0.95">
      <circle cx="50" cy="65" r="1" fill="currentColor" />
      <circle cx="50" cy="68" r="1.2" fill="currentColor" />
      <circle cx="50" cy="71" r="1" fill="currentColor" />
      <g transform="translate(42.5, 73.5) scale(0.15)">
        <path d="M 15 45 C 10 40, 8 32, 14 26 C 22 18, 34 18, 41 26 C 46 32, 42 42, 34 45...
        <path d="M 32 45 C 40 45, 52 35, 58 45 C 64 55, 52 68, 62 75" stroke-width="5.5" ...
        <path d="M 30 14 C 38 14, 45 18, 47 24" stroke-width="5" stroke-linecap="round" />
        <circle cx="43" cy="6" r="4.5" fill="currentColor" stroke="none" />
      </g>
      <line x1="50" y1="87.5" x2="50" y2="92" stroke-width="0.8" stroke-dasharray="1 1" />
      <circle cx="50" cy="94" r="1.2" fill="currentColor" />
    </g>
  </svg>`;
};

interface Testimonial {
  id: string;
  name: string;
  role: string;
  avatar: string;
  rating: number;
  quote: string;
  intention: string;
  astrology: string;
```

FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
  scriptIndex: number;
  externalLink: string;
  avatarColor: string;
}

const TESTIMONIALS_DATA: Testimonial[] = [
  {
    id: "1",
    name: "Aarav Mehta",
    role: "Cosmic Seeker",
    avatar: "AM",
    rating: 5,
    quote: "I used to feel completely blocked financially—like no matter how hard I worke...
    intention: "Wealth & Prosperity",
    astrology: "Leo Surya Sun Sign Alignment",
    scriptIndex: 1,
    externalLink: "https://lumalabs.ai/dream-machine",
    avatarColor: "from-[#C5A059] to-[#8E6E32]"
  },
  {
    id: "2",
    name: "Priya Kapoor",
    role: "Healed Soul",
    avatar: "PK",
    rating: 5,
    quote: "After losing someone close to me, I felt a heavy grief lodged in my chest. Au...
    intention: "Love & Grief Healing",
    astrology: "Venus Resonance - Cancer Moon",
    scriptIndex: 2,
    externalLink: "https://heygen.com",
    avatarColor: "from-pink-500/80 to-rose-600/80"
  },
  {
    id: "3",
    name: "Kabir Singh",
    role: "Spiritual Warrior",
    avatar: "KS",
    rating: 5,
    quote: "As an empath, I used to absorb everyone else’s chaotic energy. I felt drained...
    intention: "Spiritual Strength",
    astrology: "D1 Chart - Scorpio Ascendant Protection",
    scriptIndex: 3,
    externalLink: "https://leonardo.ai",
    avatarColor: "from-blue-600/80 to-indigo-700/80"
  },
  {
    id: "4",
    name: "Ananya Sharma",
    role: "Intentional Alchemist",
    avatar: "AS",
    rating: 5,
    quote: "Stop inking temporary names that you will regret in two years! I switched to ...
```

FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
    intention: "Soul Manifestation vs Random Ink",
    astrology: "Universal Manifestation - Higher Self Alignment",
    scriptIndex: 4,
    externalLink: "https://lumalabs.ai/dream-machine",
    avatarColor: "from-emerald-500/80 to-teal-700/80"
  }
];

export default function App() {
  // Navigation tabs
  const [activeTab, setActiveTab] = useState<"quiz" | "collection" | "premium">("quiz");

  // Premium / Business Model State
  const [activePlan, setActivePlan] = useState<"free" | "premium" | "pro" | "family">("fr...
  const isPremiumUnlocked = activePlan === "premium" || activePlan === "pro" || activePla...
  const [registeredSingleUser, setRegisteredSingleUser] = useState<{ firstName: string; l...
    try {
      const data = localStorage.getItem("aura_ink_single_user");
      return data ? JSON.parse(data) : null;
    } catch (e) {
      return null;
    }
  });
  const [familyUsersList, setFamilyUsersList] = useState<{ firstName: string; lastName: s...
    try {
      const data = localStorage.getItem("aura_ink_family_users");
      return data ? JSON.parse(data) : [];
    } catch (e) {
      return [];
    }
  });
  const [showUpgradeToFamilyModal, setShowUpgradeToFamilyModal] = useState<boolean>(false);
  const [showKundliInfoModal, setShowKundliInfoModal] = useState<boolean>(false);
  const [attemptedUser, setAttemptedUser] = useState<{ firstName: string; lastName: strin...

  // Seeker Testimonials States
  const [activeTestimonialIndex, setActiveTestimonialIndex] = useState<number>(0);

  const [unlockedIntentionsList, setUnlockedIntentionsList] = useState<string[]>(["Wealth...
  const [selectedPremiumOptionIndex, setSelectedPremiumOptionIndex] = useState<number>(0);
  const [showPremiumSimulatorModal, setShowPremiumSimulatorModal] = useState<boolean>(fal...
  const [showPrintIframeModal, setShowPrintIframeModal] = useState<boolean>(false);
  const [showDownloadPortal, setShowDownloadPortal] = useState<boolean>(false);
  const [deferredPrompt, setDeferredPrompt] = useState<any>(null);
  const [isAppInstalled, setIsAppInstalled] = useState<boolean>(false);
  const [downloadPortalTab, setDownloadPortalTab] = useState<"pwa" | "stores">("pwa");
  const [copiedCommand, setCopiedCommand] = useState<string | null>(null);

  // Privacy Policy state
  const [showPrivacyModal, setShowPrivacyModal] = useState<boolean>(false);
  const [viewState, setViewState] = useState<string | null>(() => {
    try {
```

FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
0417        const params = new URLSearchParams(window.location.search);
0418        return params.get("view");
0419      } catch (e) {
0420        return null;
0421      }
0422    });
0423
0424    useEffect(() => {
0425      const handleBeforeInstallPrompt = (e: Event) => {
0426        e.preventDefault();
0427        setDeferredPrompt(e);
0428      };
0429
0430      const handleAppInstalled = () => {
0431        setIsAppInstalled(true);
0432        setDeferredPrompt(null);
0433      };
0434
0435      window.addEventListener("beforeinstallprompt", handleBeforeInstallPrompt as any);
0436      window.addEventListener("appinstalled", handleAppInstalled);
0437
0438      return () => {
0439        window.removeEventListener("beforeinstallprompt", handleBeforeInstallPrompt as any);
0440        window.removeEventListener("appinstalled", handleAppInstalled);
0441      };
0442    }, []);
0443    const [glimpseLevel, setGlimpseLevel] = useState<"Moderate" | "Advanced" | null>(null);
0444    const [showGlimpseModal, setShowGlimpseModal] = useState<boolean>(false);
0445    const [glimpseModalLevel, setGlimpseModalLevel] = useState<"Moderate" | "Advanced" | nu...
0446    const [glimpseTimer, setGlimpseTimer] = useState<number>(0);
0447
0448    useEffect(() => {
0449      if (showGlimpseModal) {
0450        setGlimpseTimer(1);
0451      } else {
0452        setGlimpseTimer(0);
0453      }
0454    }, [showGlimpseModal]);
0455
0456    useEffect(() => {
0457      if (glimpseTimer <= 0) return;
0458      const timerId = setTimeout(() => {
0459        setGlimpseTimer((prev) => Math.max(0, prev - 1));
0460      }, 1000);
0461      return () => clearTimeout(timerId);
0462    }, [glimpseTimer]);
0463
0464    const handlePrint = () => {
0465      try {
0466        // In web browsers, running window.print() inside a sandboxed <iframe> (like the AI...
0467        // is blocked by standard iframe security rules. We check if we are in an iframe an...
0468        const isIframe = window.self !== window.top;
```

FIRST 10 PAGES OF CORE SOURCE CODE (SRC/APP.TSX) • APPLICATION STATE ENGINE, IMPORTS, TYPES & ARCHETYPES

```
0469        if (isIframe) {
0470          setShowPrintIframeModal(true);
0471        } else {
0472          window.print();
0473        }
0474      } catch (err) {
0475        setShowPrintIframeModal(true);
0476      }
0477    };
0478
0479    // Form State
0480    const [firstName, setFirstName] = useState("Aara");
0481    const [lastName, setLastName] = useState("Sharma");
0482    const [gender, setGender] = useState("Female");
0483    const [birthDate, setBirthDate] = useState("1995-07-12");
0484    const [birthTime, setBirthTime] = useState("20:30");
0485    const [dontKnowExactTime, setDontKnowExactTime] = useState<boolean>(false);
0486    const [approxTimePeriod, setApproxTimePeriod] = useState<string>("unknown");
0487    const [birthPlace, setBirthPlace] = useState("London, England, United Kingdom");
0488    const [showMapModal, setShowMapModal] = useState(false);
0489    const [showBirthPlaceDropdown, setShowBirthPlaceDropdown] = useState(false);
0490    const [intention, setIntention] = useState("Wealth");
0491    const [need, setNeed] = useState("I feel blocked in my financial stability and want to ...
0492    const [quizStep, setQuizStep] = useState<1 | 2 | 3 | 4>(1);
0493    const [preferredLanguage, setPreferredLanguage] = useState<string>("English");
0494
0495    // App UI State
0496    const [loading, setLoading] = useState(false);
0497    const [errorMsg, setErrorMsg] = useState<string | null>(null);
0498    const [isCopied, setIsCopied] = useState(false);
0499    const [activeResonancePoint, setActiveResonancePoint] = useState<string>("ankle");
0500
0501    // Preview states for body mockups
0502    const [previewGender, setPreviewGender] = useState<"Female" | "Male">("Female");
0503    const [inkColor, setInkColor] = useState<"Charcoal" | "Gold" | "BlueBlack">("Charcoal");
0504    const [selectedComplexity, setSelectedComplexity] = useState<"Simple" | "Moderate" | "A...
0505
0506    // Voice recognition states
0507    const [isListening, setIsListening] = useState(false);
0508    const [speechError, setSpeechError] = useState<string | null>(null);
0509
0510    const toggleListening = () => {
0511      const SpeechRecognition = (window as any).SpeechRecognition || (window as any).webkit...
0512      if (!SpeechRecognition) {
0513        setSpeechError("Voice to text is not supported in this browser. Try Chrome or Safar...
0514        setTimeout(() => setSpeechError(null), 4000);
0515        return;
0516      }
0517
0518      if (isListening) {
0519        if ((window as any)._activeRecognition) {
0520          try {
```